ТЗ: ВИПРАВЛЕННЯ ОПЛАТИ TELEGRAM STARS, WEB INVOICE І TON WALLET

Проєкт: StorkCrypto
Дата: 21.10.2026
Платформа: Telegram Mini App + зовнішній web-доступ
Статус проблеми: платежі не запускаються або не завершуються; wallet picker не закривається після підключення; застосунок не показує підтверджений wallet address.

---

1. Критично важливе правило платформи

PRO-підписка StorkCrypto є цифровою послугою всередині Telegram. Для продажу цифрових товарів і послуг Telegram вимагає використовувати Telegram Stars з currency `XTR` [1].

Тому не можна реалізовувати таку логіку:

```
Plain Text

Telegram Mini App → PRO subscription → оплата GRAM/TON замість Stars
```

Це не є правильною заміною Telegram payment flow.

Правильний поділ:

```
Plain Text

Telegram Mini App:
PRO digital subscription → Telegram Stars (XTR)

TON Wallet:
підключення TON-гаманця, дозволені blockchain-функції,
окремі TON-транзакції або окремі дозволені сервіси

Зовнішній web:
можна відкрити Telegram invoice link,
але не можна активувати PRO лише на підставі frontend redirect
```

TON Connect дозволений для підключення TON-гаманців і blockchain-взаємодій, але це окремий механізм, а не заміна Stars для цифрової підписки [2] [3].

---

2. Виправити Telegram Stars у Mini App

2.1. Backend endpoint створення invoice

Створити або виправити server-side endpoint:

```
Plain Text

POST /api/payments/stars/create
Authorization: Telegram initData
Content-Type: application/json
```

Request:

```
JSON

{
  "plan": "pro_monthly"
}
```

Backend повинен:

1. Перевірити Telegram `initData` через HMAC.
2. Отримати справжній Telegram user id із валідованих даних.
3. Не приймати `user_id` як довірене поле з frontend.
4. Визначити ціну на сервері, а не приймати її від клієнта.
5. Створити унікальний `invoice_payload`.
6. Використати currency `XTR`.
7. Для цифрової послуги не використовувати GRAM як currency.
8. Повернути invoice link або invoice slug.

Приклад server-side параметрів:

```
Plain Text

{
  title: "StorkCrypto PRO",
  description: "Доступ до PRO-функцій StorkCrypto на 30 днів",
  payload: signedPayload,
  currency: "XTR",
  prices: [
    { label: "PRO Monthly", amount: PRO_PRICE_XTR }
```

```
  ],
  provider_token: ""
}
```

Для Stars Telegram вказує, що `provider_token` для digital goods може бути порожнім [1]. Не передавати bot token у frontend.

2.2. Frontend відкриття invoice

У Telegram Mini App після відповіді backend:

Plain Text

```
const tg = window.Telegram?.WebApp;

const response = await fetch('/api/payments/stars/create', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `tma ${tg?.initData ?? ''}`,
  },
  body: JSON.stringify({ plan: 'pro_monthly' }),
});

const { invoiceUrl } = await response.json();

if (!invoiceUrl) {
  throw new Error('Invoice URL was not returned');
}

tg.openInvoice(invoiceUrl, (status) => {
  // status is informational only.
  // It is NOT proof of payment.
  refreshSubscriptionFromServer();
});
```

Telegram Mini Apps мають використовувати `WebApp.openInvoice(url, callback)`, а після закриття invoice отримують статус події invoice [4]. Callback не можна вважати server-side підтвердженням.

2.3. Обов’язкова server-side обробка платежу

Telegram надсилає боту:

1. `pre_checkout_query`;

2. після успішної оплати — `successful_payment`.

`pre_checkout_query` потрібно підтвердити протягом 10 секунд, інакше транзакція скасовується [1].

Псевдологіка webhook:

Plain Text

```
if (update.pre_checkout_query) {
  const q = update.pre_checkout_query;

  const order = await db.orders.findByPayload(q.invoice_payload);

  const valid =
    order &&
    order.user_id === q.from.id &&
    q.currency === 'XTR' &&
    q.total_amount === order.amount &&
    order.status === 'PENDING';

  await telegram.answerPreCheckoutQuery(
    q.id,
    valid,
    valid ? undefined : 'Order validation failed'
  );
}

if (update.message?.successful_payment) {
  const payment = update.message.successful_payment;

  const order = await db.orders.findByPayload(payment.invoice_payload);

  if (!order) {
    await logSecurityEvent('UNKNOWN_PAYMENT_PAYLOAD', payment);
    return;
  }

  if (payment.currency !== 'XTR') return;

  if (payment.total_amount !== order.amount) return;

  if (payment.telegram_payment_charge_id already exists) {
    return; // idempotent duplicate webhook
  }

  await db.transaction(async (tx) => {
    await tx.payments.insert({
```

```
      order_id: order.id,
      user_id: update.message.from.id,
      currency: payment.currency,
      amount: payment.total_amount,
      telegram_charge_id: payment.telegram_payment_charge_id,
      provider_charge_id: payment.provider_payment_charge_id ?? null,
      status: 'PAID',
    });

    await tx.subscriptions.activate({
      user_id: update.message.from.id,
      plan: order.plan,
      source: 'TELEGRAM_STARS',
      payment_id: payment.telegram_payment_charge_id,
      starts_at: new Date(),
      expires_at: addDays(new Date(), 30),
    });
  });
}
```

Telegram прямо вимагає видавати digital goods/services тільки після отримання `successful_payment`; саме підтвердження `pre_checkout_query` не доводить успішність оплати `1`.

2.4. Джерело істини для PRO

Frontend не має права самостійно встановлювати:

Plain Text

```
isPro = true
```

Після invoice callback frontend повинен лише оновити дані:

Plain Text

```
GET /api/me/subscription
```

Backend повертає:

JSON

```
{
  "plan": "pro",
  "status": "active",
  "source": "TELEGRAM_STARS",
```

```
  "expiresAt": "2026-11-20T10:00:00.000Z"
}
```

До отримання цього server response користувач залишається BASIC.

---

3. Web-оплата поза Telegram

3.1. Рекомендований web-сценарій

Якщо користувач відкрив сайт у звичайному браузері, не потрібно намагатися імітувати Telegram invoice всередині сайту.

Після натискання:

Plain Text

```
Оплатити через Telegram
```

web повинен:

1. Запросити на backend одноразовий invoice link.
2. Backend створює invoice через `createInvoiceLink` із currency `XTR`.
3. Web відкриває отриманий HTTPS/Telegram link.
4. Telegram app або Telegram Web відкриває invoice.
5. Після оплати backend отримує `successful_payment`.
6. Користувач повертається на web.
7. Web опитує backend за статусом order, а не довіряє query parameter.

Схема:

Plain Text

```
Web → POST /api/payments/stars/create-link
   → Telegram invoice link
   → user opens Telegram
   → pays with XTR
   → bot webhook verifies successful_payment
   → web polls /api/payments/:orderId/status
   → PRO active only after backend confirmation
```

3.2. Web fallback

Якщо Telegram не встановлений або invoice не відкривається:

```
Plain Text
Відкрити Telegram для завершення оплати
```

Показати кнопку:

```
Plain Text
Open in Telegram
```

Не показувати повідомлення «Оплата успішна» лише після redirect.

Не передавати `?success=true` як доказ платежу.

3.3. Важливе обмеження

Зовнішній web checkout не повинен використовуватися для обходу правил Telegram щодо цифрових послуг у Mini App. Для PRO, який надається всередині Telegram, основним і безпечним шляхом залишити Stars `XTR` [1].

Якщо окремо потрібен web-план для користувачів поза Telegram, його потрібно оформити як окремий billing product із власними Terms, account linking, refund logic і способом підтвердження особи. Не змішувати його з Telegram Stars order.

---

4. GRAM / TON

4.1. Не називати GRAM валютою Telegram Stars

`XTR` — Telegram Stars.

`TON` або native Gram — blockchain asset у TON Network.

Це різні системи. Не можна передавати `GRAM` у поле `currency` Telegram Stars invoice.

4.2. Якщо потрібна TON-оплата

Реалізовувати її лише окремим потоком:

```
Plain Text
```

```
TON Wallet Connect
→ користувач підписує транзакцію
→ backend отримує order nonce
→ backend перевіряє on-chain transaction
→ перевіряє recipient, amount, comment/payload, lt/hash
→ встановлює TON_PAYMENT_CONFIRMED
```

Не активувати Telegram PRO від одного лише:

- підключення гаманця;
- wallet address;
- натискання Pay;
- відкриття TonConnect modal;
- frontend transaction callback.

Окремо перевірити, чи конкретна TON payment model відповідає правилам Telegram для Mini Apps і цифрових послуг. Якщо немає юридично та платформно безпечного обґрунтування — залишити TON Wallet лише для connection/дозволених blockchain-взаємодій, а PRO продавати через Stars.

---

5. Виправити TON Connect wallet picker

5.1. Причина описаної проблеми

Після вибору гаманця меню не зникає, тому що найімовірніше:

- створено кілька `TonConnectUI` instances;
- custom wallet modal не слухає `onStatusChange`;
- provider монтується повторно при зміні вкладки;
- overlay закривається лише кнопкою, а не після connect event;
- wallet state не синхронізується з React state;
- UI використовує старий account state;
- відсутній public `tonconnect-manifest.json` або він недоступний для wallet.

5.2. Правильна React-структура

Встановити:

Bash

```
npm install @tonconnect/ui-react
```

У root один раз:

TSX

```
<TonConnectUIProvider
  manifestUrl="https://storkcrypto.pages.dev/tonconnect-manifest.json"
>
  <App />
</TonConnectUIProvider>
```

Не монтувати Provider всередині Portfolio, modal або кожного render.

У Portfolio:

TSX

```
import {
  TonConnectButton,
  useTonAddress,
  useTonWallet,
  useIsConnectionRestored,
} from '@tonconnect/ui-react';

function WalletStatus() {
  const address = useTonAddress();
  const wallet = useTonWallet();
  const restored = useIsConnectionRestored();

  if (!restored) {
    return <span>Restoring wallet session…</span>;
  }

  if (!wallet) {
    return (
      <div>
        <TonConnectButton />
        <span>TON Wallet not connected</span>
      </div>
    );
  }

  return (
    <div>
      <span>TON Wallet connected</span>
```

```
      <code>{shortenTonAddress(address)}</code>
      <TonConnectButton />
    </div>
  );
}
```

Офіційний TON Connect UI автоматично перемикає кнопку між `Connect Wallet` і connected account state [5] [6].

5.3. Якщо залишається власна обгортка

Використати один `onStatusChange`:

Plain Text

```
useEffect(() => {
  return tonConnectUI.onStatusChange((wallet) => {
    if (wallet) {
      setWalletConnected(true);
      setWalletAddress(wallet.account.address);
      setWalletName(wallet.device.appName);
      setWalletPickerOpen(false);
    } else {
      setWalletConnected(false);
      setWalletAddress(null);
      setWalletName(null);
      setWalletPickerOpen(false);
    }
  });
}, [tonConnectUI]);
```

Після `wallet !== null` обов’язково:

Plain Text

```
setWalletPickerOpen(false);
```

Не залишати старий custom overlay у DOM.

5.4. Показ адреси

Показувати friendly address із маскуванням:

Plain Text

```
UQAb12…xYz9
```

Додати:

- Copy address;
- Open in explorer;
- Disconnect;
- Connected wallet name;
- network status;
- error/retry state.

Не показувати повний address у великому тексті на мобільному екрані.

5.5. Manifest

Файл повинен бути доступний без авторизації:

Plain Text

```
https://storkcrypto.pages.dev/tonconnect-manifest.json
```

Вміст:

JSON

```
{
  "url": "https://storkcrypto.pages.dev",
  "name": "StorkCrypto",
  "iconUrl": "https://storkcrypto.pages.dev/icon-180.png",
  "termsOfUseUrl": "https://storkcrypto.pages.dev/terms",
  "privacyPolicyUrl": "https://storkcrypto.pages.dev/privacy"
}
```

Перевірити:

Plain Text

```
GET /tonconnect-manifest.json → 200
GET /icon-180.png → 200
Access-Control-Allow-Origin: * або доступ без CORS-block
Content-Type: application/json
HTTPS
```

TON Connect вимагає публічно доступний HTTPS manifest; при проблемах із manifest виникають `MANIFEST_NOT_FOUND_ERROR` або `MANIFEST_CONTENT_ERROR` [5].

5.6. Чому wallet picker має кілька варіантів

Сам список Telegram Wallet, Tonkeeper, MyTonWallet або іншого TON Connect-сумісного wallet — нормальна частина wallet picker. Його не потрібно прибирати до вибору.

Після успішного connect повинно відбутися:

```
Plain Text

wallet picker closes
→ app receives wallet state
→ app shows “TON Wallet connected”
→ app shows shortened address
→ Connect button changes to connected/disconnect state
```

Якщо список залишається після connection, це вже помилка lifecycle/overlay state.

---

6. Підписка та UI states

У PRO modal додати стани:

```
Plain Text

BASIC
Invoice preparing…
Waiting for Telegram payment…
Payment verification…
PRO active
Payment canceled
Payment failed
Payment pending
```

Після натискання `Pay`:

- disable повторні clicks;
- показати spinner;
- створити client request id;
- backend створити idempotent order;
- відкрити invoice;

- після закриття не показувати success до server polling;
- timeout через 60–90 секунд;
- кнопка Retry створює новий order або безпечно продовжує pending order.

---

7. Безпека

Перевірити і виправити:

- Telegram bot token тільки server-side;
- payment secret тільки server-side;
- Supabase service role тільки server-side;
- `isPro` не з localStorage;
- wallet address не є доказом payment;
- `invoice_payload` підписаний або збережений у database;
- charge id має unique constraint;
- webhook має idempotency;
- initData HMAC перевіряється;
- redirect URL allowlist;
- немає open redirect;
- немає `window.open(untrustedUrl)`;
- зовнішні посилання мають `noopener,noreferrer`;
- не логувати payment secrets або initData в console.

---

8. Acceptance tests

Telegram Stars

Plain Text

```
1. Відкрити Mini App у Telegram.
2. Натиснути Upgrade to Pro.
3. Перевірити, що відкривається Telegram invoice.
4. До успішної оплати PRO не активується.
```

```
5. Натиснути Cancel — статус залишається BASIC.
6. Завершити тестову Stars payment у Telegram test environment.
7. Перевірити pre_checkout_query.
8. Перевірити successful_payment.
9. Перевірити запис payment у database.
10. Перевірити PRO після reload.
11. Повторити webhook — дубль не створюється.
```

Web

Plain Text

```
1. Відкрити сайт у Chrome.
2. Натиснути Pay via Telegram.
3. Відкрити invoice link.
4. Якщо Telegram відсутній — показати Open in Telegram.
5. Не показувати success лише після redirect.
6. Після оплати web перевіряє backend order status.
```

TON Wallet

Plain Text

```
1. Відкрити Portfolio.
2. Натиснути Connect TON Wallet.
3. Wallet picker відкривається.
4. Вибрати wallet.
5. Picker автоматично закривається.
6. В app видно Connected.
7. В app видно скорочений address.
8. Copy address працює.
9. Disconnect працює.
10. Reload відновлює session або чесно показує Restoring/Disconnected.
11. Зміна вкладки не створює другий picker.
12. 30 циклів connect/disconnect не залишають orphan overlay.
```

Regression

Plain Text

```
/api/chat → HTTP 200 з валідною відповіддю
/api/validate без initData → HTTP 400
```

```
Console → немає uncaught payment/wallet/chunk errors
Mobile 360×800 → немає горизонтального overflow
Fullscreen modal → відкривається поверх навігації
Loading screen → зберігається вертикальний брендований Manifesto на 6,5 с
```

---

9. Заборонені спрощення

Не робити:

- `PRO = true` після click;
- `PRO = true` після `invoiceClosed` зі статусом `paid` без backend check;
- `PRO = true` після підключення wallet;
- показувати GRAM як Telegram currency;
- замінювати Stars на TON для Telegram digital subscription;
- залишати wallet picker після connect;
- приховувати wallet address;
- створювати кілька TonConnect providers;
- зберігати payment proof лише у localStorage;
- показувати фальшивий success при невідомому статусі;
- додавати фіктивну транзакцію для демонстрації.

---

10. Формат фінального звіту AI-агента

Plain Text

```
Виправлено:
- ...

Telegram Stars:
- create invoice: PASS/FAIL
- XTR: PASS/FAIL
- pre_checkout_query: PASS/FAIL
- successful_payment: PASS/FAIL
- server-side PRO activation: PASS/FAIL
- duplicate webhook protection: PASS/FAIL

Web invoice:
```

```
- invoice link: PASS/FAIL
- open Telegram fallback: PASS/FAIL
- backend status polling: PASS/FAIL

TON Connect:
- manifest: PASS/FAIL
- wallet picker: PASS/FAIL
- picker closes after connect: PASS/FAIL
- address shown: PASS/FAIL
- disconnect: PASS/FAIL
- restore session: PASS/FAIL

Залишкові ризики:
- ...

Фінальний статус:
READY / NOT READY
```

Статус `READY` заборонено ставити, якщо не підтверджено server-side `successful_payment` і коректне закриття TON Connect picker.

References

[1] Telegram Bot Payments API for Digital Goods and Services
[2] TON Connect overview
[3] Telegram Blockchain Guidelines
[4] Telegram Mini Apps Web Apps documentation
[5] TON Connect get started
[6] TON Connect wallet connection and account state

11. ОБОВ'ЯЗКОВІ DESIGN INVARIANTS

Платіжні та wallet-виправлення не повинні руйнувати затверджену дизайн-концепцію StorkCrypto.

11.1. Роль AI-агента

Працюй як **Chief Design Developer**. Окрім функціонального виправлення, перевіряй читабельність, ієрархію, spacing, mobile viewport, контраст і сучасність інтерфейсу. Не додавай випадкових яскравих кольорів або декоративних елементів, які суперечать crypto-terminal стилю.

Основна естетика:

- темна;
- професійна;
- футуристична;
- серйозна;
- мінімалістична;
- не багатокольорова;
- з яскравими, але контрольованими акцентами.

Назва `StorkCrypto` у головному інтерфейсі повинна бути компактною, яскравою і не займати надмірне місце. Використовувати printing/monospace або інший технічно доречний шрифт. Не робити великий строкатий логотип.

11.2. Loading Screen — Manifesto не замінювати

Loading screen є впізнаваною частиною продукту. Заборонено повністю замінювати його новим spinner, стандартним progress screen або іншою концепцією.

Зберегти:

- вертикальну композицію;
- темний branded background;
- середній темп анімації;
- серйозний crypto/blockchain настрій;
- тривалість приблизно 6,5 секунди;
- плавний перехід у головний dashboard;
- відсутність semicircle disappearance у фіналі.

Дозволено лише покращення наявної версії:

1. `StorkCrypto` має з’являтися з реалістичного розсіюваного туману/диму.
2. Літери можуть легенько фрагментуватися.
3. Пікселі повинні відділятися поступово, pixel-by-pixel.
4. Пікселі повинні бути чітко видимими, але не створювати шум.
5. Пікселі повинні контрольовано падати вниз у loading bar.
6. Ефект має йти послідовно від одного краю слова до іншого.
7. Фінал може містити мінімалістичний geometric crypto/blockchain particle motif.

8. Допускається стриманий data spiral/DNA або holographic branded element, якщо він не виглядає випадковим чи надто абстрактним.
9. Не використовувати надто швидку, хаотичну або дитячу анімацію.
10. Не змінювати текст, login bar чи основну послідовність без необхідності.

`StorkCrypto` на mobile повинен залишатися на одному рядку. Якщо текст не поміщається, адаптувати font-size/letter-spacing, а не переносити його на два рядки.

Loading acceptance criteria

Plain Text

```
Hard reload із очищеним cache
Loading screen залишається вертикальним
StorkCrypto поміщається на одному рядку у 360×800
Fog → text emergence → fracture → pixels → loading bar → dashboard
Пікселі реально видно
Фінальний semicircle disappearance відсутній
Перехід не має blank screen і різкого flash
Фактична тривалість: 6,5 с ± 100 мс
prefers-reduced-motion не ламає логіку переходу
```

11.3. Main interface після loading

Після завершення loading animation головний dashboard повинен з’являтися плавно, без різкого jump або зміни всього кольорового простору.

Перевірити:

- темний цілісний background;
- відсутність надмірної multicolor-візуалізації;
- compact bright StorkCrypto branding;
- достатній контраст дрібного тексту;
- рівномірні card paddings;
- відсутність horizontal overflow;
- коректний safe-area відступ у Telegram WebView;
- нижню навігацію, яка не перекриває контент.

Не додавати надмірні gradient, rainbow accents або випадкові neon colors. Один основний accent і один secondary accent достатні.

11.4. Читабельність

Мінімально збільшити загальний розмір тексту там, де це не руйнує layout. Основні market indicators збільшити приблизно на 10%.

Перевірити:

- `$2.1T Market Cap`;
- `+3.2% 24h Change`;
- ціни активів;
- статуси `LIVE`, `DELAYED`, `STALE`, `DEMO`, `ERROR`;
- кнопки `Pay`, `Connect Wallet`, `Disconnect`;
- тексти модалок;
- Academy cards;
- нижню навігацію.

Icon-only controls повинні мати visible або accessible label. Не використовувати дрібний текст, який у Telegram Android WebView неможливо прочитати.

11.5. Регресійна заборона

Під час виправлення payment і wallet не можна:

- змінювати loading duration;
- повністю редизайнити Manifesto;
- прибирати pixel/fog concept;
- робити назву StorkCrypto великою або багатокольоровою;
- повертати semicircle disappearance;
- перетворювати dark interface на білий dashboard;
- зменшувати шрифти, щоб приховати overflow;
- ховати payment/wallet error замість коректного повідомлення;
- показувати фальшиве `Payment successful` або `Wallet connected`.

11.6. Фінальна design-перевірка

Перед статусом `READY` зробити screenshots і перевірити:

```
Plain Text
```

```
Loading screen: 360×800, 390×844, 412×915
Dashboard Day: 360×800, 390×844
Dashboard Night: 360×800, 390×844
PRO payment modal: fullscreen/compact exception працює правильно
TON wallet connected state: адреса, Disconnect, статус
TON wallet picker: зникає після підключення
Academy UA/EN/RU: контент реально змінюється
```

Якщо дизайн став функціональним, але втратив dark non-multicolor StorkCrypto identity, статус має залишатися `NOT READY` до виправлення.